오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

HTML/CSS

DAY 05

CSS 박스 모델과 레이아웃

요소의 크기와 여백이 어떻게 계산되는지 이해합니다.
Flexbox와 Grid로 실제 화면 배치를 직접 만들어 봅니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1. **요소의 크기가 어떻게 계산되는지 설명할 수 있다**

   content, padding, border, margin의 관계를 이해합니다.

2. **display와 position으로 요소를 배치할 수 있다**

   줄바꿈 여부와 기준점을 골라 원하는 자리에 놓습니다.

3. **Flexbox와 Grid로 실제 레이아웃을 만들 수 있다**

   내비게이션 바와 카드 목록을 직접 구성합니다.

오늘의 결과물

내비게이션 바와 카드 세 개가 나란히 놓인 페이지

ROADMAP

오늘의 흐름

01

박스 모델

네 겹 구조
box-sizing

02

display

줄바꿈 여부
block과 inline

03

position

기준점 고르기
float와 clear

04

Flexbox

한 줄 배치
실습 01

실습 포함

05

Grid

행과 열
실습 02

실습 포함

박스 모델을 먼저 이해해야 배치가 왜 그렇게 되는지 설명할 수 있습니다.
Flexbox와 Grid는 오늘 이후 모든 화면 작업에서 계속 씁니다.

나흘 동안 만든 것

1	**DAY 01 · 문서 구조**	html·head·body와 제목, 문단, 목록, 이미지, 링크
2	**DAY 02 · 폼**	action·method·name, input type, 기본 검증
3	**DAY 03 · 시맨틱**	header·nav·main·section·article·footer
4	**DAY 04 · CSS 선택자**	style.css를 연결하고 원하는 요소를 골라 색을 입혔습니다
5	**지금 상태**	색은 입혔지만 배치는 아직 정하지 않았습니다

오늘은 그 요소들을 원하는 자리에 놓습니다

되짚기 · 02

지금 style.css는 이런 상태입니다

```
* { margin: 0; padding: 0; }
h1 { color: #0b6fb8; }
.card { border: 1px solid #d3dee6; padding: 20px; }
nav > a { color: #1b3f5e; }

/* margin과 padding은 어제 적었지만
   그 값이 정확히 무엇인지는 오늘 배웁니다 */
```

오늘 채우는 것

요소의 크기와 여백,
그리고 놓이는 자리입니다.
속성 이름으로는 width, padding,
margin, display, position입니다.

선택자는 그대로 씁니다

어제 배운 선택자로 요소를 고르고
오늘 배우는 속성을 적습니다.

준비

오늘의 셋팅

```
Day05/
  about-me.html    ← Day04에서 복사해 옵니다
  style.css        ← Day04에서 복사해 옵니다
  boxmodel.html    ← 박스 모델 확인용으로 새로 만듭니다
  me.jpg

/* style.css 맨 위 * 규칙에 한 줄을 더합니다 */
* { box-sizing: border-box; margin: 0; padding: 0; }

/* boxmodel.html 은 상자 하나만 있는 파일입니다 */
```

확인하고 시작합니다

- 파일 네 개가 같은 폴더에
- about-me.html에 색이 적용된다
- 개발자 도구를 열 수 있다

오늘 계속 열어둘 것

- about-me.html 브라우저 창
- 개발자 도구 Computed 탭

01

박스 모델

네 겹 구조 · 크기 계산 · box-sizing

01 · 문제

200px이라고 적었는데 200px이 아닙니다

```
.card {
  width: 200px;
  padding: 20px;
  border: 5px solid #d3dee6;
}

/* 화면에서 재보면 250px */
```

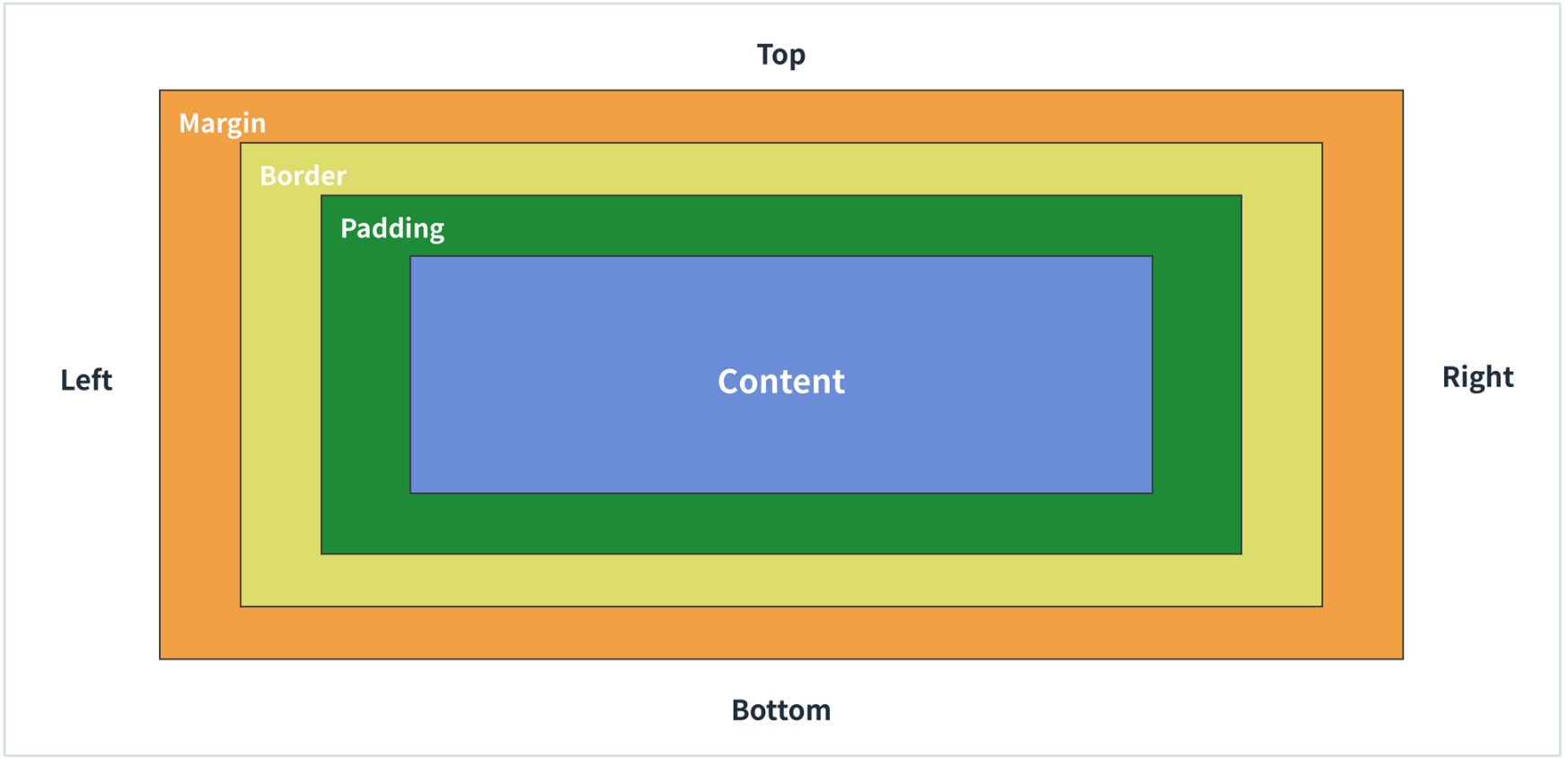
왜 이런 일이 생기나요

width가 상자 전체가 아니라
내용 영역만 가리키기 때문입니다.
여백과 테두리는 그 바깥에
따로 더해집니다.

그래서 먼저 배웁니다

박스 모델을 알면 이 계산이
바로 설명됩니다.

요소 하나는 이렇게 네 겹으로 되어 있습니다



안쪽부터 content → padding → border → margin. 네 겹 모두 위·아래·왼쪽·오른쪽 값을 따로 가집니다.

모든 요소는 네 겹의 상자입니다

`content`	글자나 이미지가 들어가는 실제 내용
`padding`	내용과 테두리 사이의 안쪽 여백
`border`	상자를 둘러싼 테두리
`margin`	다른 요소와 벌어지는 바깥 여백

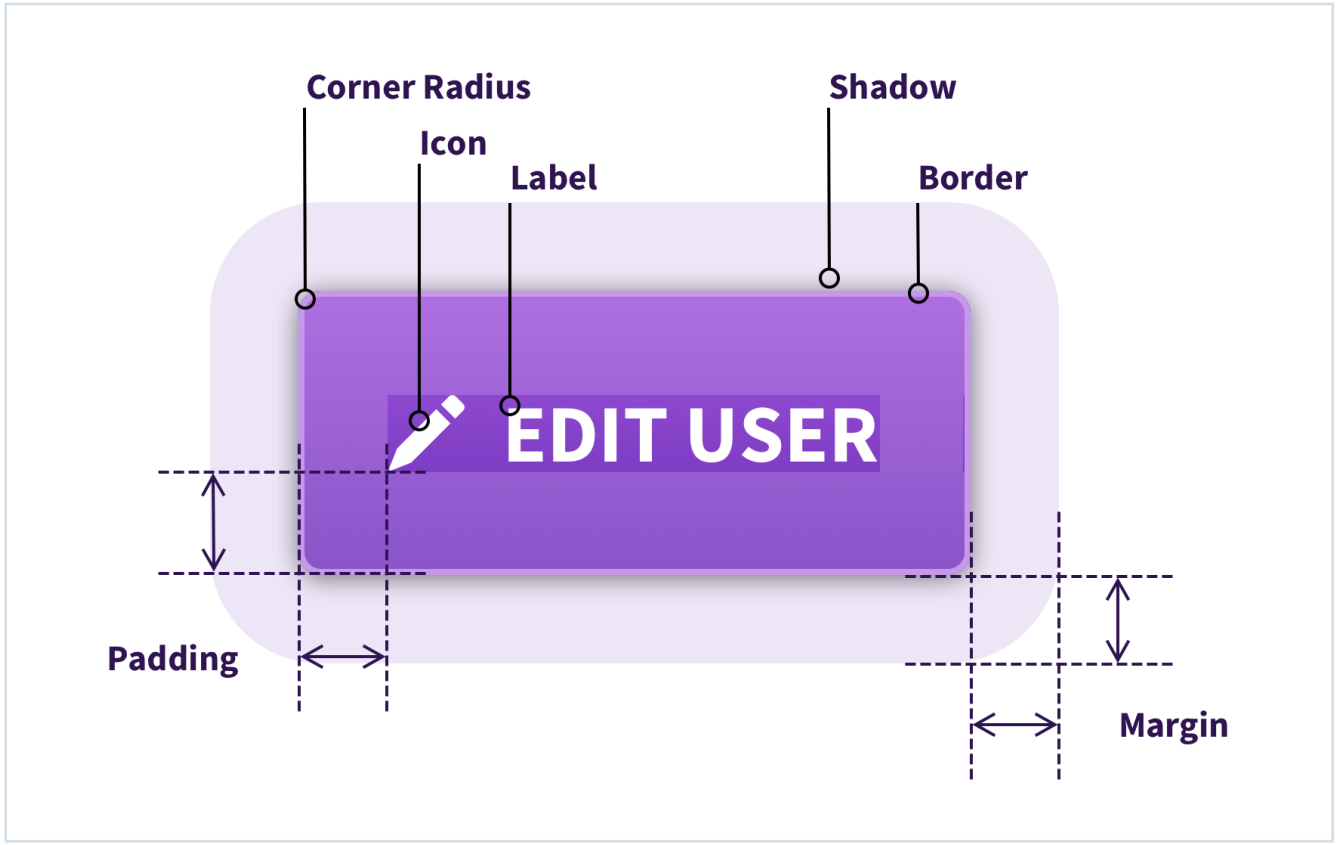
바깥에서 안쪽 순서

margin이 가장 바깥이고
content가 가장 안쪽입니다.
개발자 도구에서 요소를 선택하면
이 네 겹이 그대로 보입니다.

네 겹은 항상 있습니다

값을 적지 않았을 뿐이지
모든 요소가 이 구조입니다.
브라우저가 기본값을 갖고 있습니다.

버튼 하나를 뜯어보면 이렇습니다



버튼을 구성하는 요소에 이름을 붙인 그림

우리가 배운 이름이 보입니다

글자 주위의 여백이 padding,
테두리가 border,
바깥 여백이 margin입니다.
가운데 글자 자리가 content입니다.

나머지도 CSS로 정합니다

모서리 둥글기는 border-radius,
그림자는 box-shadow로 정합니다.
border-radius는 오늘 다룹니다.

content — 내용이 들어가는 자리

```
.box {
  width: 200px;
  height: 120px;
}

/* width와 height는
   이 영역의 크기입니다 */
```

기본값은 auto입니다

width를 적지 않으면 block 요소는
부모의 가로를 다 차지하고
height는 내용만큼만 생깁니다.

글이 넘치면

height를 고정한 상자는
내용이 밖으로 삐져나옵니다.
높이는 되도록 고정하지 않습니다.

padding — 내용과 테두리 사이의 안쪽 여백

```
padding: 20px;                /* 네 방향 모두 */
padding: 10px 20px;           /* 위아래 / 좌우 */
padding: 10px 20px 30px 40px;
                /* 위 오른쪽 아래 왼쪽 */

padding-top: 10px;
padding-left: 40px;
```

값 네 개는 시계 방향

위에서 시작해 오른쪽,
아래, 왼쪽 순서입니다.

배경색이 칠해집니다

padding 영역까지는 배경색이
들어갑니다. 카드 안쪽이
넓어 보이는 이유입니다.

border — 상자를 둘러싼 테두리

```
border: 1px solid #d3dee6;
        굵기  종류   색

border-radius: 8px;   /* 모서리 둥글게 */
border-bottom: 2px solid #0b6fb8;

/* 종류 : solid, dashed, dotted, none */
```

세 값을 한 줄에 적습니다

굵기, 종류, 색을 띄어쓰기로
적습니다. 순서는 바뀌어도 됩니다.
종류를 빠뜨리면 안 보입니다.

border-radius

모서리를 둥글게 만듭니다.
정사각형에 50%를 주면 원이 됩니다.

margin — 다른 요소와 벌어지는 바깥 여백

```
margin: 20px;
margin: 20px auto;    /* 위아래 20px, 좌우 자동 */
margin-bottom: 40px;

.container {
  width: 960px;
  margin: 0 auto;     /* 가로 가운데 정렬 */
}
```

auto는 남는 공간을 나눕니다

좌우에 auto를 주면 남는 자리를
양쪽이 똑같이 나눠 갖습니다.
그래서 가운데로 옵니다.

배경색이 칠해지지 않습니다

margin은 상자 바깥이라
배경이 비어 보입니다.

위아래 margin은 더해지지 않습니다

예상

20 + 30 = 50px

```
.a { margin-bottom: 20px; }
.b { margin-top: 30px; }
```

사이가 50px 벌어질 것 같습니다.

실제

큰 쪽만 남아 30px

```
/* 실제로는 30px */
```

보통 흐름의 세로 방향에서만 일어납니다.

여백이 이상하면 위아래 요소의 margin을 함께 봅니다

가로 방향은 그대로 더해집니다. flex와 grid의 자식 사이에서는 상쇄가 일어나지 않습니다.

width가 무엇의 너비인지가 문제입니다

content-box — 기본값

실제 너비 250px

```
width: 200px;
padding: 20px;
border: 5px solid;
/* 200 + 20*2 + 5*2 = 250 */
```

width는 내용 영역만 가리킵니다.

border-box

실제 너비 200px

```
box-sizing: border-box;
width: 200px;  padding: 20px;
border: 5px solid;
/* 내용 영역이 150px로 줄어듭니다 */
```

width가 테두리까지 포함합니다.

border-box에서는 적은 width가 그대로 화면의 너비입니다

01 · 실무

어제 만든 * 규칙에 한 줄을 더합니다

```
* {
  box-sizing: border-box;    /* 오늘 추가 */
  margin: 0;
  padding: 0;
}

/* style.css 맨 위에 둡니다 */
```

어제 적은 리셋

margin과 padding을 0으로 맞춰
브라우저 기본 여백을 없앴습니다.
맨 위에서 기본값을 지우는 규칙을
리셋이라고 부릅니다.

오늘 더하는 한 줄

box-sizing까지 같이 바꿔두면
크기 계산이 계속 편합니다.

01 · 확인

개발자 도구에서 네 겹을 눈으로 봅니다

boxmodel.html을 열고 상자를 우클릭한 뒤 검사를 누릅니다.
Computed 탭 맨 위에 박스 모델 그림이 있습니다.

그림에서 읽는 순서

- 가장 안쪽 숫자가 content 크기
- 그 바깥이 padding
- 그 바깥이 border
- 가장 바깥이 margin

함께 해볼 것

- box-sizing 주석을 풀고 저장하기
- 그림의 숫자가 바뀌는지 보기
- 상자를 두 개로 늘려 margin 확인

레퍼런스 MDN — 박스 모델

여기까지 정리

1	**네 겹**	안쪽부터 content, padding, border, margin
2	**padding과 margin**	안쪽 여백은 배경이 칠해지고 바깥 여백은 비어 보입니다
3	**기본 계산**	width는 내용 영역만 가리키고 여백과 테두리가 더해집니다
4	**border-box**	테두리까지 포함해서 세므로 적은 값이 그대로 나옵니다
5	**margin 상쇄**	보통 흐름에서 위아래로 놓인 여백은 큰 값 하나만 남습니다

다음은 이 상자들을 어떻게 늘어놓을지입니다

02

display

block · inline · inline-block · none

display는 줄을 바꿀지 말지를 정합니다

```
<p>문단입니다</p>
<p>다음 문단입니다</p>
/* 위아래로 쌓입니다 */

<span>글자</span><span>글자</span>
/* 옆으로 이어집니다 */

span { display: block; }
/* 바꿀 수 있습니다 */
```

태그마다 기본값이 있습니다

p와 div는 block,
span과 a는 inline이 기본값입니다.
이 값은 CSS로 바꿀 수 있습니다.

오늘 다루는 값

block, inline, inline-block, none과
flex, grid까지 여섯 개입니다.

block — 새 줄에서 시작하고 가로를 다 씁니다

```
div, p, h1, section { display: block; }

.box {
  width: 200px;     /* 적용됩니다 */
  height: 120px;    /* 적용됩니다 */
  margin: 20px 0;   /* 위아래 적용됩니다 */
}
```

대표 태그

div, p, h1부터 h6,
section, article, header,
footer, ul

쓰는 곳

문단과 섹션처럼
덩어리를 나눌 때 씁니다.

inline — 글자의 흐름을 따라 옆으로 이어집니다

```
span, a, strong, em { display: inline; }

span {
  width: 200px;     /* 무시됩니다 */
  height: 100px;    /* 무시됩니다 */
  padding: 20px;    /* 좌우만 자리를 만듭니다 */
}
```

크기를 줄 수 없습니다

글자 크기만큼만 자리를
차지합니다.

위아래 padding과 margin

padding은 배경만 칠해지고
다른 줄을 밀어내지 못해 겹칩니다.
위아래 margin은 무시됩니다.

inline-block — 옆으로 이어지고 크기도 갖습니다

```
.btn {
  display: inline-block;
  width: 120px;     /* 적용됩니다 */
  padding: 12px 0; /* 적용됩니다 */
  text-align: center;
}
```

두 성질을 합친 값

줄을 바꾸지 않으면서
width와 height를 가질 수 있습니다.
버튼이나 입력창에 자주 씁니다.

태그 사이 공백이 남습니다

HTML의 줄바꿈이 띄어쓰기로 남아
요소 사이에 틈이 생깁니다.
지금은 Flexbox로 해결합니다.

02 · none

숨기는 방법은 두 가지입니다

display: none

자리까지 없어집니다

```
.menu { display: none; }

/* 화면에서 사라지고
   차지하던 자리도 없어집니다 */
```

뒤 요소가 그 자리를 채웁니다.

visibility: hidden

자리는 남습니다

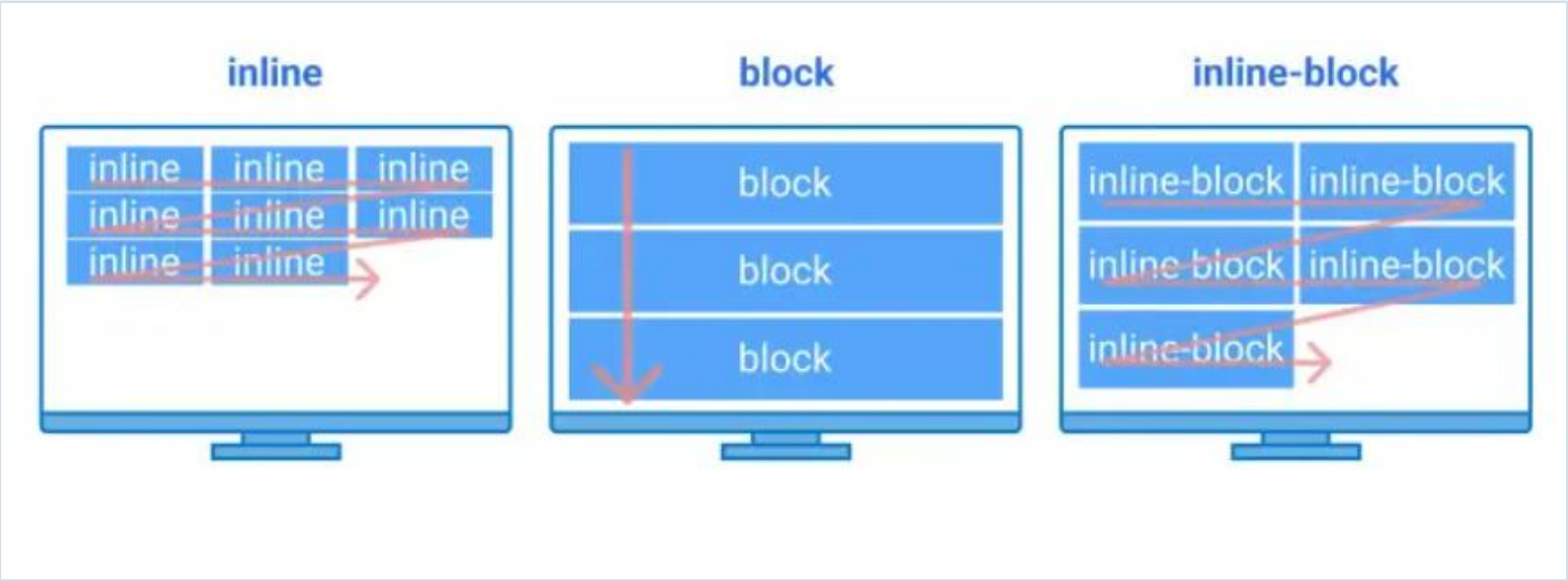
```
.menu { visibility: hidden; }

/* 보이지 않지만
   자리는 그대로 비어 있습니다 */
```

뒤 요소는 움직이지 않습니다.

메뉴를 여닫거나 화면 크기에 따라 감출 때 display: none을 씁니다

세 값이 화면에서 어떻게 다른지 봅니다



inline은 옆으로 이어지고, block은 줄을 차지하며, inline-block은 옆으로 이어지면서 크기를 갖습니다.

줄바꿈 여부와 크기 지정 가능 여부, 이 두 가지로 갈립니다

02 · 비교

네 값을 한눈에

값	줄바꿈	크기 지정	대표 태그
block	새 줄에서 시작	width·height 가능	div, p, h1, section
inline	옆으로 이어짐	width·height 무시 (img 등 제외)	span, a, strong
inline-block	옆으로 이어짐	width·height 가능	button, input
none	화면에서 사라짐	자리도 차지하지 않음	숨길 때 지정

크기를 주고 싶은데 안 먹으면 그 요소가 inline인지부터 봅니다

inline-block이나 block으로 바꾸면 width와 height가 바로 적용됩니다.

쌓기와 배치의 차이

block

위에서 아래로 쌓입니다

- 새 줄에서 시작하고 부모 가로를 다 씀
- 자식의 배치를 직접 정할 수 없음
- width, height, margin, padding으로 조절

flex

부모가 자식을 배치합니다

- 자식을 행이나 열로 늘어놓음
- 정렬과 간격을 부모가 정함
- flex-direction, justify-content, gap

고정된 구조는 block, 정렬이 필요한 배치는 flex

오늘 만들 내비게이션 바처럼 요소를 나란히 놓고 간격을 맞춰야 하면 flex를 씁니다.

02 · 예제

값을 바꿔가며 눈으로 확인합니다

강의 저장소의 display.html을 열어 값을 하나씩 바꿔 봅니다.

예제 강의 저장소 — display.html

이 순서로 바꿔 봅니다

- span에 width 주기 — 변화 없음
- span을 inline-block으로 바꾸기
- 다시 block으로 바꾸기

확인할 것

- 줄이 바뀌는지
- 가로를 다 쓰는지
- 크기가 적용되는지

세 값을 번갈아 적용해 보면 표를 외우지 않아도 차이가 몸에 남습니다.

03

position

기준점 고르기 · float와 clear

position은 무엇을 기준으로 놓을지 정합니다

```
.box {
  position: relative;
  top: 20px;
  left: 40px;
}

/* top, right, bottom, left와
   함께 써야 움직입니다 */
```

다섯 가지 값

static, relative, absolute,
fixed, sticky입니다.
기본값은 static입니다.

혼자서는 아무 일도 없습니다

relative는 top이나 left를
함께 적어야 움직입니다.
absolute와 fixed는 적는 순간
흐름에서 빠집니다.

static — 기본값, 코드에 적은 순서대로

```
.box { position: static; }

/* 아무것도 적지 않은 상태와 같습니다 */

.box {
  position: static;
  top: 50px;   /* 무시됩니다 */
}
```

지금까지 만든 화면

모든 요소가 static이었습니다.
HTML에 적은 순서대로
위에서 아래로 놓입니다.

top과 left가 무시됩니다

static인 요소는 위치 값을 적어도
움직이지 않습니다.

레퍼런스 MDN — position

relative — 원래 자리를 기준으로 밀어냅니다

```
.box {
  position: relative;
  top: 20px;     /* 아래로 20px */
  left: 40px;    /* 오른쪽으로 40px */
}

/* 원래 자리는 그대로 비어 있습니다 */
```

자리는 남습니다

보이는 위치만 옮겨지고
차지하던 자리는 그대로입니다.
다른 요소는 움직이지 않습니다.

더 중요한 쓰임

absolute 자식의 기준점이 됩니다.
다음 장에서 이어집니다.

absolute — 기준이 되는 조상을 찾아 붙습니다

```
.card {
  position: relative;   /* 기준점 */
}
.badge {
  position: absolute;
  top: 10px;
  right: 10px;   /* 카드 오른쪽 위 */
}
```

가장 가까운 조상

position이 static이 아닌
조상을 위로 찾아 올라가
그 상자를 기준으로 붙습니다.

자리를 차지하지 않습니다

문서 흐름에서 빠지므로
뒤 요소가 그 자리를 채웁니다.

fixed — 화면을 기준으로 고정됩니다

```css
.top-btn {
  position: fixed;
  right: 24px;
  bottom: 24px;
}

/* 스크롤해도 같은 자리에 남습니다 */
```

기준은 보이는 화면

부모가 아니라 브라우저 화면을
기준으로 자리를 잡습니다.

자주 쓰는 곳

맨 위로 가기 버튼,
항상 보이는 상단 바,
화면을 덮는 팝업

sticky — 흐르다가 정해진 지점에서 멈춥니다

```
.nav {
  position: sticky;
  top: 0;   /* top·bottom·left·right 중 하나 */
}

/* 평소에는 흐름을 따라가다가
   화면 맨 위에 닿으면 붙습니다 */
```

멈출 지점을 적어야 합니다

top·bottom·left·right 중 하나는 적어야 합니다. 없으면 아무 일도 일어나지 않습니다.

부모 안에서만 붙어 있습니다

부모 영역을 벗어나면 다시 흐름을 따라 올라갑니다.

가장 많이 쓰는 조합은 이 한 쌍입니다

```
<div class="card">
  <img src="me.jpg" alt="프로필">
  <span class="badge">NEW</span>
</div>

.card  { position: relative; }
.badge { position: absolute; top: 8px; right: 8px; }
```

예제 강의 저장소 — position.html

읽는 순서

부모에 relative를 주어
기준을 만들고,
자식에 absolute를 주어
그 안의 원하는 자리에 붙입니다.

쓰이는 곳

카드 위의 라벨,
이미지 위의 설명,
버튼 옆의 알림 숫자

float은 읽을 줄만 알면 됩니다

원래 목적

글이 이미지를 감싸게

신문처럼 이미지 옆으로
글이 흘러가게 만드는 속성입니다.
clear는 그 흐름을 끊습니다.

```
img { float: left; }
```

지금은

새 화면에는 쓰지 않습니다

한동안 전체 배치에 쓰였기 때문에
오래된 코드에는 남아 있습니다.
새로 만들 때는 Flexbox를 씁니다.

```
.next { clear: both; }
```

float은 읽고 이해하는 수준까지만 합니다

04

Flexbox

두 개의 축 · 정렬 속성 · 실습 01

부모에 한 줄 적으면 자식이 나란히 놓입니다

```
<ul class="menu">
  <li>소개</li>
  <li>작업</li>
  <li>연락</li>
</ul>

.menu { display: flex; }
/* 세로로 쌓이던 li가 가로로 눕습니다 */
```

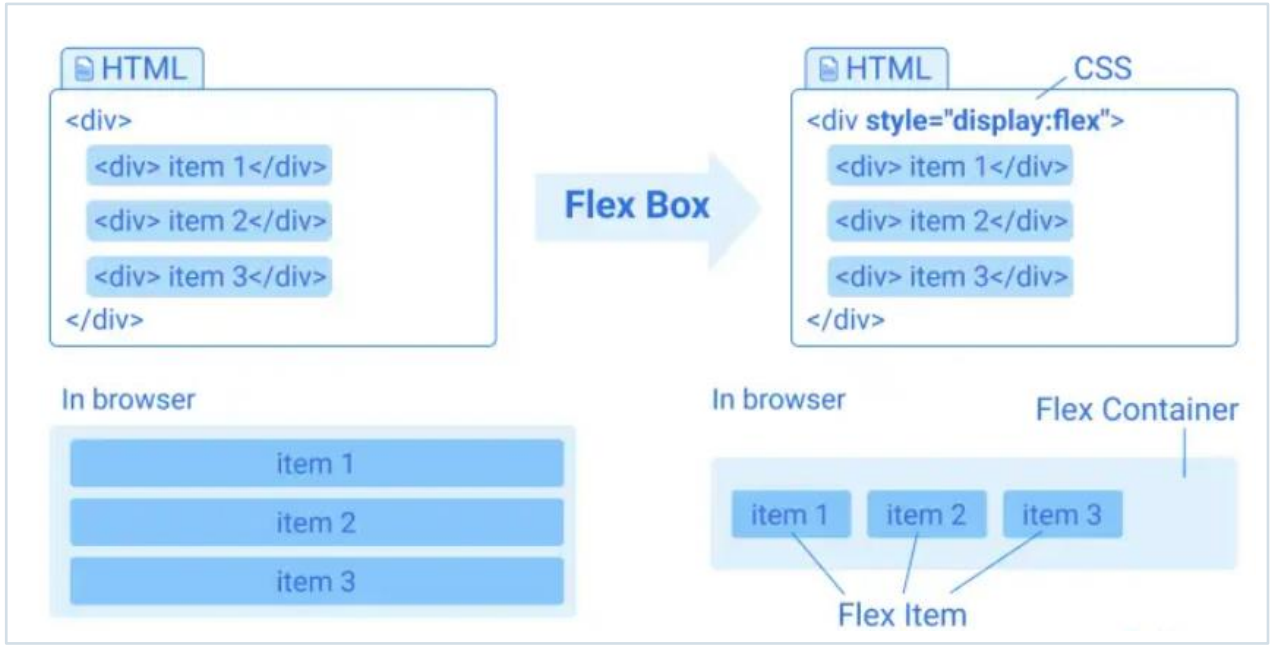
부모에 줍니다

자식에 주면 그 자식의 내용만 flex로 배치됩니다. 나란히 놓고 싶은 요소의 부모에 줘야 합니다.

자식은 그대로 둡니다

li에는 아무것도 적지 않습니다. 부모가 배치를 정합니다.

태그 구조는 그대로, CSS 한 줄로 배치가 바뀝니다



왼쪽은 flex가 없을 때, 오른쪽은 부모에 display: flex를 준 뒤

부모가 flex 컨테이너, 자식들이 flex 아이템이 됩니다

두 개의 축으로 생각합니다

주축 main axis

늘어놓는 방향

flex-direction이 정하는 방향입니다.
기본값 row에서는 가로가 주축입니다.
justify-content가 이 축을 정렬합니다.

교차축 cross axis

주축과 직각인 방향

주축이 가로면 교차축은 세로입니다.
align-items가 이 축을 정렬합니다.
높이를 맞출 때 쓰는 축입니다.

이름이 헷갈리면 방향부터 확인합니다

justify는 늘어놓은 방향, align은 그와 직각인 방향이라고 기억하면 됩니다.

flex-direction — 어느 방향으로 늘어놓을지

```
.menu { flex-direction: row; }
/* 기본값. 왼쪽에서 오른쪽 */

.menu { flex-direction: column; }
/* 위에서 아래로 */

/* row-reverse, column-reverse 도 있습니다 */
```

축이 함께 바뀝니다

column으로 바꾸면 주축이 세로가 됩니다. 그러면 justify-content가 세로를 정렬하게 됩니다.

반응형의 출발점

넓은 화면에서는 row, 좁은 화면에서는 column으로 바꾸는 방식을 자주 씁니다.

justify-content — 주축 방향으로 모으고 벌립니다

```
flex-start      /* 앞쪽으로 모음, 기본값 */
center          /* 가운데로 모음 */
flex-end        /* 뒤쪽으로 모음 */
space-between   /* 양 끝에 붙이고 사이를 균등하게 */
space-around    /* 각 요소 양옆에 같은 여백 */
space-evenly    /* 사이와 양 끝 여백을 모두 같게 */

.nav { justify-content: space-between; }

/* 같은 요소 셋을 값만 바꿔 보면 */
flex-start      [A][B][C]............
center          ......[A][B][C]......
space-between   [A]......[B]......[C]
```

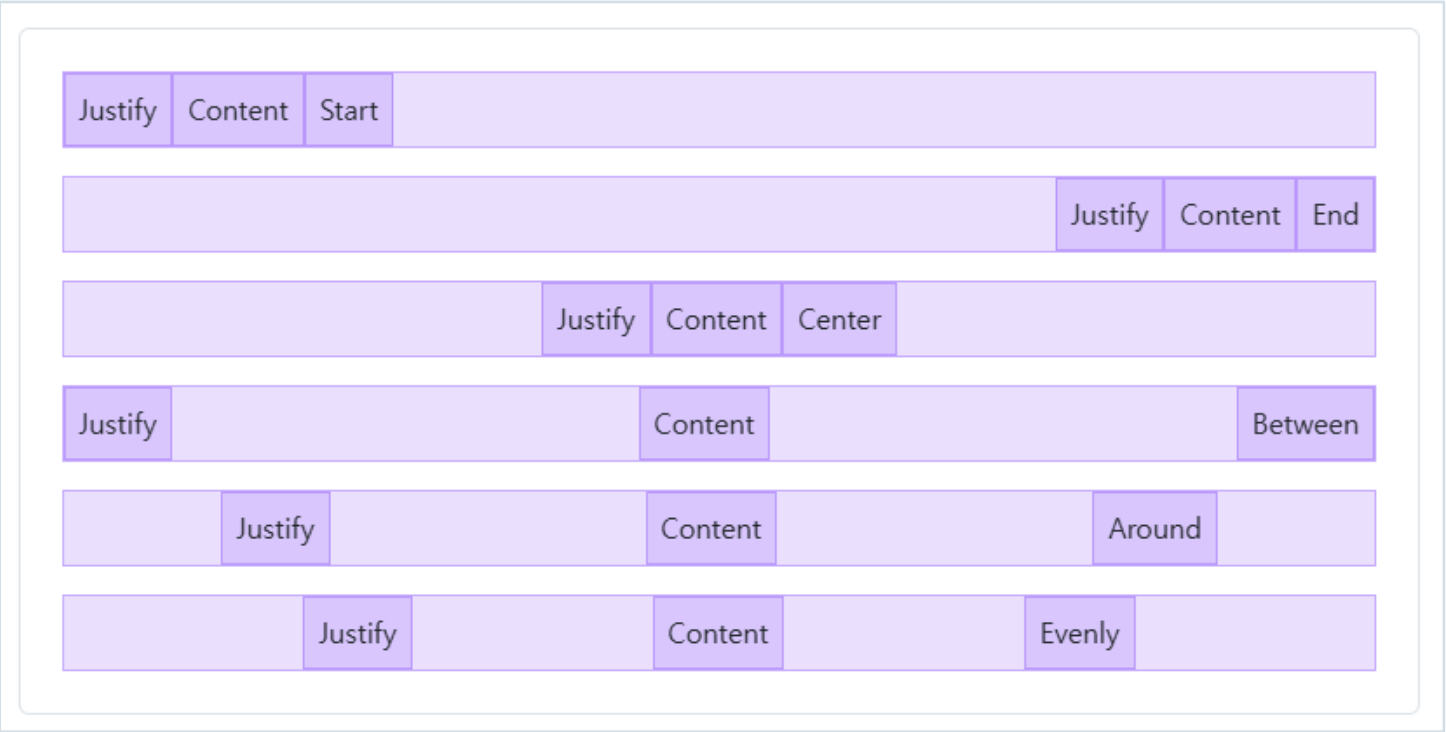
가장 많이 쓰는 값

space-between입니다.
로고는 왼쪽, 메뉴는 오른쪽에
붙이는 배치가 여기서 나옵니다.

실습 01에서 바로 씁니다

내비게이션 바의 양 끝 배치를
이 값 하나로 만듭니다.

여섯 개 값을 나란히 놓고 봅니다



위에서부터 flex-start · flex-end · center
space-between · space-around · space-evenly

읽는 법

보라색 막대가 flex 아이템입니다.
값에 따라 막대가 어디로 모이는지만
보면 됩니다.

space- 세 가지의 차이

between은 양 끝에 붙이고,
around는 각 요소 양옆에,
evenly는 모든 간격을 같게 둡니다.

align-items — 교차축 방향으로 줄을 맞춥니다

```
stretch     /* 높이를 늘려 채움, 기본값 */
center      /* 가운데 맞춤 */
flex-start  /* 위쪽 맞춤 */
flex-end    /* 아래쪽 맞춤 */

.nav { align-items: center; }
```

세로 가운데 정렬

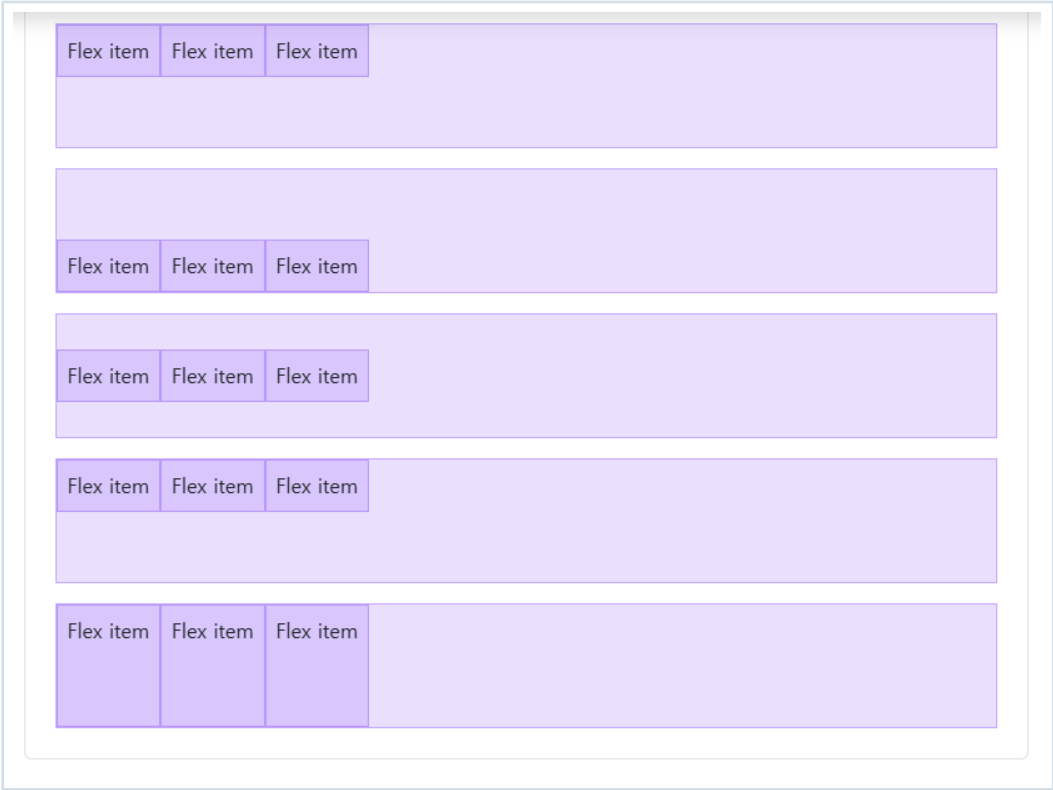
로고와 메뉴의 글자 크기가 달라도
align-items: center 한 줄로
줄이 맞습니다.

예전에는 어려웠습니다

세로 가운데 정렬은 Flexbox 전에는
쉽지 않은 작업이었습니다.

04 · align-items

교차축 정렬도 그림으로 확인합니다



위에서부터 flex-start · flex-end · center
baseline · stretch

기본값은 stretch

아무것도 적지 않으면 자식들이
부모 높이만큼 늘어납니다.
카드 높이가 저절로 맞는 이유입니다.

center를 가장 많이 씁니다

로고와 메뉴처럼 크기가 다른
요소의 줄을 맞출 때 씁니다.

간격을 주고 넘칠 때 줄을 바꿉니다

```
.menu {
  display: flex;
  gap: 16px;          /* 자식 사이 간격 */
  flex-wrap: wrap;    /* 모자라면 다음 줄로 */
}

.item { flex: 1; }    /* 자식들의 너비가 같아집니다 */
```

gap은 부모에

자식마다 margin을 주지 않아도 사이 간격이 한 번에 정해집니다.

flex는 자식에

flex: 1은 flex: 1 1 0의 줄임입니다. 기준 크기를 0으로 두고 남는 공간을 똑같이 나눕니다. 내용을 살리려면 1 1 auto.

04 · 정리

자주 쓰는 여섯 개

속성	어디에	하는 일
flex-direction	부모	row는 가로, column은 세로로 늘어놓음
justify-content	부모	주축 방향 정렬, space-between이 특히 유용
align-items	부모	교차축 방향 정렬, center로 세로 가운데 맞춤
flex-wrap	부모	자리가 모자라면 다음 줄로 넘길지 결정
gap	부모	자식들 사이의 간격
flex	자식	공간을 나눠 갖는 비율. flex: 1이면 똑같이 나눔

여섯 개만 알아도 실제 화면의 대부분을 배치할 수 있습니다

04 · 연습

Flexbox는 게임으로 익히는 편이 빠릅니다

속성 이름을 외우기보다 값을 바꿔보며 결과를 보는 편이 훨씬 빨리 남습니다.

연습 Flexbox Froggy — 개구리를 옮기며 배우는 flex 연습

이렇게 진행합니다

- 강사 화면에 띄우고 함께 읽기
- 값을 먼저 말해보게 하기
- 틀리면 어느 축인지 되묻기

여기까지 나오는 것

- justify-content와 align-items
- flex-direction
- order와 flex-wrap

레퍼런스 MDN — Flexbox 안내서

실습 01

내비게이션 바 만들기

about-me.html의 nav를 가로 메뉴로 바꿉니다.
로고는 왼쪽, 메뉴는 오른쪽에 붙도록 배치합니다.

```
<nav class="nav">
  <span class="logo">김코딩</span>
  <ul class="menu">
    <li><a href="#intro">소개</a></li>
    <li><a href="#work">작업</a></li>
  </ul>
</nav>
.nav  { display: flex; justify-content: space-between; }
.menu { display: flex; gap: 16px; list-style: none; }
.menu a { color: #1b3f5e; text-decoration: none; }  /* nav > a 대신 */
```

한 줄씩 적고 저장하며 확인합니다

list-style → flex → gap → 부모 flex → space-between → .menu a

확인할 것

- 로고와 메뉴가 양 끝에 있다
- 메뉴가 가로로 놓인다
- 목록 점이 사라졌다

막히면

display: flex를 어디에 줬는지
먼저 확인합니다.

실습 01 · 점검

가로로 안 눕는다면

01

어디에 줬는지

display: flex를 li나 a에
주지 않았는지 봅니다.
ul에 줘야 합니다.
부모에 주는 속성입니다.

02

규칙이 닿았는지

선택자 이름이 HTML의
class와 같은지 봅니다.
.menu와 menu는 다릅니다.
점 하나를 자주 빠뜨립니다.

03

저장하고 새로고침

style.css를 저장했는지,
브라우저를 새로고침했는지
확인합니다.
가장 흔한 원인입니다.

개발자 도구에서 ul을 선택하면 flex 표시가 붙습니다

그 표시가 없으면 규칙이 적용되지 않은 것이므로 선택자부터 다시 봅니다.

05

Grid

행과 열 · fr 단위 · 실습 02

행과 열로 나눠 배치합니다

```
.cards {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
}

/* 자식이 몇 개든 세 칸씩 채워집니다 */
```

칸을 먼저 정합니다

몇 칸으로 나눌지 부모가 정하고
자식이 순서대로 들어갑니다.

넘치면 다음 줄로

칸 수를 넘는 자식은
자동으로 다음 행에 놓입니다.

05 · fr

fr은 남는 공간을 나누는 단위입니다

```
grid-template-columns: 1fr 1fr 1fr;
/* 셋으로 똑같이 나눕니다 */

grid-template-columns: 2fr 1fr;
/* 2대 1로 나눕니다 */

grid-template-columns: 200px 1fr;
/* 왼쪽은 고정, 오른쪽은 나머지 전부 */
```

px과 섞어 쓸 수 있습니다

사이드바는 고정 폭,
본문은 나머지 전부인 배치를
두 줄로 만들 수 있습니다.

gap은 여기서도 같습니다

행과 열 사이 간격을
한 번에 정합니다.

05 · 선택

Flexbox와 Grid는 이렇게 나눠 씁니다

Flexbox

한 줄로 늘어놓을 때

내비게이션 바, 버튼 묶음,
아이콘과 글자를 나란히 놓기.
개수가 정해져 있지 않아도 됩니다.

Grid

행과 열을 함께 다룰 때

카드 목록, 사진 갤러리,
페이지 전체 골격 나누기.
칸 수를 미리 정해둡니다.

한 화면에서 같이 써도 됩니다

카드 목록 전체는 Grid로 나누고, 카드 하나 안쪽은 Flexbox로 정렬하는 식으로 섞어 씁니다.

실습 02

카드 세 개를 나란히 놓기

작업 영역의 카드를 세 개로 늘려 div로 묶고, Grid로 나란히 배치합니다.
카드 안쪽 여백과 테두리는 박스 모델에서 배운 속성으로 정합니다.

```
<div class="cards">
  <article class="card"><h3>프로젝트 A</h3><p>설명</p></article>
  <article class="card"><h3>프로젝트 B</h3><p>설명</p></article>
  <article class="card"><h3>프로젝트 C</h3><p>설명</p></article>
</div>

.cards { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 20px; }
.card  { padding: 20px; border: 1px solid #d3dee6; border-radius: 8px; }
```

이 순서로 적습니다

- display: grid 만 주면 아직 한 열입니다
- grid-template-columns 를 주면 셋이 나란히 놓입니다

확인할 것

- 세 카드의 너비가 같다
- 카드 사이 간격이 같다
- 안쪽 여백이 들어갔다

다 했다면 카드를 하나 더

연습 Grid Garden

점검

레이아웃에서 자주 나오는 세 가지

!

크기를 줬는데 커져요

padding과 border가
밖으로 더해진 경우입니다.
box-sizing을 확인합니다.

!

flex가 안 먹어요

display: flex를 자식에
준 경우가 많습니다.
부모에 줘야 합니다.

!

absolute가 엉뚱하게 붙어요

absolute의 기준이 되는
조상이 없어서입니다.
부모에 relative를 줍니다.

배치가 뜻대로 안 되면 개발자 도구에서 그 요소를 선택합니다

박스 모델 그림에서 어느 겹이 커졌는지 보면 원인이 대부분 바로 드러납니다.

WRAP UP

오늘 정리

개념

- 모든 요소는 네 겹의 상자로 되어 있다
- border-box는 테두리까지 포함해 크기를 센다
- 보통 흐름에서 위아래 margin은 큰 값 하나만 남는다
- display는 줄바꿈 여부를 정한다
- position은 기준점을 정한다
- Flexbox는 한 줄, Grid는 격자다

직접 쓸 수 있어야 하는 것

- padding과 margin 구분해 쓰기
- box-sizing으로 크기 계산 맞추기
- display 값 바꿔가며 배치하기
- relative와 absolute 짝으로 쓰기
- flex로 가로 메뉴 만들기
- grid로 카드 나란히 놓기

NEXT

DAY 06

반응형 웹과 HTML/CSS 종합 실습

오늘 만든 카드는 화면을 줄이면 좁아지기만 합니다.
다음 시간에는 미디어 쿼리로 화면 크기에 따라 배치를 바꾸고,
1챕터를 마무리하며 개인 소개 페이지를 완성합니다.

수고하셨습니다.